CLOUD COMPUTING

ACTIVITY – PARTITIONING

Study and Implementation of Partitioning and Indexing Strategies using PostgreSQL and Cassandra

NAME: Asmi Vishal Kapadnis

SRN: PES2UG23CS100

SECTION: B

AIM:

To understand and implement different data partitioning strategies such as:

- Key-Range Partitioning (PostgreSQL)
- Hash Partitioning using Consistent Hashing (Cassandra)
- Secondary Indexing Strategies (Scatter/Gather and Global Index)

OBJECTIVES:

- To observe how range partitioning distributes data
- To identify the hot spot problem
- To understand how hash partitioning ensures uniform distribution
- To compare scatter/gather vs global indexing

PHASE 1: Key-Range Partitioning (PostgreSQL)

1. Created a partitioned table
2. Created partitions
3. Inserted 1000 records with same timestamp using Python script.
4. When 1000 records with the same timestamp were inserted, all records were stored in a single partition (readings_q1). The EXPLAIN output confirmed that only one partition was accessed. This demonstrates the hot spot problem, where one partition becomes overloaded while others remain idle.

PHASE 2: Hash Partitioning (Cassandra)

1. Created keyspace
2. Created table
3. Inserted sample records
4. Queried data
5. In Cassandra, the partition key (sensor_id) is hashed using the Murmur3 hashing algorithm. This ensures that data is distributed across the cluster rather than being grouped together. Hence, unlike range partitioning, no hot spot is created.

PHASE 3: Secondary Indexing Strategies

1. Scatter/Gather (PostgreSQL)
    a) Added new column
    b) Updated values
    c) Created index
    d) Query
    e) The EXPLAIN output shows that PostgreSQL performs index scans on multiple partitions (readings_q1 and readings_q2). This indicates a scatter/gather approach, where the query is executed on all partitions and results are combined. This increases query overhead.
2. Global Index (Manual Table)
    a) Created table
    b) Populated table
    c) Queried
    d) The global index table allows direct lookup based on sensor_type without scanning all partitions. This improves read performance but requires additional updates during insert operations, increasing write cost.

OBSERVATIONS:

```
pes2ug23cs100@pes2ug23cs100:~/CC Lab/partitioning_lab$ docker compose up -d
WARN[0000] /home/pes2ug23cs100/CC Lab/partitioning_lab/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] up 31/31
 ✔ Image postgres:15                          Pulled
 ✔ Image cassandra:latest                     Pulled
 ✔ Network partitioning_lab_default           Created
 ✔ Container partitioning_lab-pg_node-1       Created
 ✔ Container partitioning_lab-cassandra_node-1 Created
pes2ug23cs100@pes2ug23cs100:~/CC Lab/partitioning_lab$ docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED          STATUS         PORTS                                                                                  NAMES
28848ec78905   postgres:15        "docker-entrypoint.s…"   18 seconds ago   Up 17 seconds  0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp                                            partitioning_lab-pg_node-1
20b9b8e76d57   cassandra:latest   "docker-entrypoint.s…"   18 seconds ago   Up 17 seconds  7000-7001/tcp, 7199/tcp, 9160/tcp, 0.0.0.0:9042->9042/tcp, [::]:9042->9042/tcp         partitioning_lab-cassandra_node-1
```

1. Phase 1:

```
pes2ug23cs100@pes2ug23cs100:~/CC Lab/partitioning_lab$ docker exec -it partitioning_lab-pg_node-1 psql -U postgres
psql (15.17 (Debian 15.17-1.pgdg13+1))
Type "help" for help.

postgres=# \dt
                  List of relations
 Schema |      Name       |       Type        |  Owner
--------+-----------------+-------------------+----------
 public | readings_q1     | table             | postgres
 public | readings_q2     | table             | postgres
 public | sensor_readings | partitioned table | postgres
 public | user_blobs      | table             | postgres
 public | user_metadata   | table             | postgres
(5 rows)
```

```
pes2ug23cs100@pes2ug23cs100:~/CC Lab/partitioning_lab$ python3 app/experiment.py
Query Plan Target: Seq Scan on readings_q1 sensor_readings  (cost=0.00..31.25 rows=8 width=20)
```

- All records were inserted into a single partition (readings_q1)
- Demonstrated uneven data distribution

2. Phase 2:

```
pes2ug23cs100@pes2ug23cs100:~/CC Lab/partitioning_lab$ docker exec -it partitioning_lab-cassandra_node-1 cqlsh
Connected to Test Cluster at 127.0.0.1:9042
[cqlsh 6.2.0 | Cassandra 5.0.6 | CQL spec 3.4.7 | Native protocol v5]
Use HELP for help.
cqlsh> SOURCE '/setup.cql';
cqlsh> DESCRIBE KEYSPACE lab_hashing;

CREATE KEYSPACE lab_hashing WITH replication = {'class': 'SimpleStrategy', 'replication_factor': '1'}  AND durable_writes = true;

CREATE TABLE lab_hashing.sensor_hashes (
    sensor_id uuid PRIMARY KEY,
    data_value float,
    location text
) WITH additional_write_policy = '99p'
    AND allow_auto_snapshot = true
    AND bloom_filter_fp_chance = 0.01
    AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
    AND cdc = false
    AND comment = ''
    AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy', 'max_threshold': '32', 'min_threshold': '4'}
    AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
    AND memtable = 'default'
    AND crc_check_chance = 1.0
    AND default_time_to_live = 0
    AND extensions = {}
    AND gc_grace_seconds = 864000
    AND incremental_backups = true
    AND max_index_interval = 2048
    AND memtable_flush_period_in_ms = 0
    AND min_index_interval = 128
    AND read_repair = 'BLOCKING'
    AND speculative_retry = '99p';
cqlsh> USE lab_hashing;
cqlsh:lab_hashing> INSERT INTO sensor_hashes (sensor_id, location, data_value) VALUES (uuid(), 'Chennai', 25.5);
cqlsh:lab_hashing> INSERT INTO sensor_hashes (sensor_id, location, data_value) VALUES (uuid(), 'Delhi', 30.2);
cqlsh:lab_hashing> INSERT INTO sensor_hashes (sensor_id, location, data_value) VALUES (uuid(), 'Mumbai', 28.7);
cqlsh:lab_hashing> SELECT * FROM sensor_hashes;

 sensor_id                            | data_value | location
--------------------------------------+------------+----------
 3c386f72-e7a8-479b-ba36-4dd81d9e0809 |       28.7 |   Mumbai
 ea619f26-1b59-4c0c-bab6-4795998e6b80 |       30.2 |    Delhi
 c43e47fc-3494-41cb-872b-8bea820e9061 |       25.5 |  Chennai

(3 rows)
cqlsh:lab_hashing> quit
```

- Data distributed using hash of partition key
- Even distribution ensured (conceptually)

3. Phase 3:

- Scatter/Gather: Query accessed multiple partitions

```
pes2ug23cs100@pes2ug23cs100:~/CC Lab/partitioning_lab$ docker exec -it partitioning_lab-pg_node-1 psql -U postgres
psql (15.17 (Debian 15.17-1.pgdg13+1))
Type "help" for help.

postgres=# ALTER TABLE sensor_readings ADD COLUMN sensor_type TEXT;
ALTER TABLE
postgres=# UPDATE sensor_readings SET sensor_type = 'temperature';
UPDATE 1000
postgres=# CREATE INDEX idx_sensor_type ON sensor_readings(sensor_type);
CREATE INDEX
postgres=# EXPLAIN SELECT * FROM sensor_readings WHERE sensor_type = 'temperature';
                                        QUERY PLAN
-------------------------------------------------------------------------------------------
 Append  (cost=4.19..28.30 rows=10 width=52)
   ->  Bitmap Heap Scan on readings_q1 sensor_readings_1  (cost=4.19..15.59 rows=5 width=52)
         Recheck Cond: (sensor_type = 'temperature'::text)
         ->  Bitmap Index Scan on readings_q1_sensor_type_idx  (cost=0.00..4.19 rows=5 width=0)
               Index Cond: (sensor_type = 'temperature'::text)
   ->  Bitmap Heap Scan on readings_q2 sensor_readings_2  (cost=4.19..12.66 rows=5 width=52)
         Recheck Cond: (sensor_type = 'temperature'::text)
         ->  Bitmap Index Scan on readings_q2_sensor_type_idx  (cost=0.00..4.19 rows=5 width=0)
               Index Cond: (sensor_type = 'temperature'::text)
(9 rows)
```

- Global Index: Query accessed only one table

```
postgres=# CREATE TABLE global_sensor_index (sensor_type TEXT, sensor_id INT);
CREATE TABLE
postgres=# INSERT INTO global_sensor_index SELECT sensor_type, sensor_id FROM sensor_readings;
INSERT 0 1000
postgres=# SELECT * FROM global_sensor_index WHERE sensor_type = 'temperature';
```

pes2ug23cs100@pes2ug23cs100: ~/CC Lab/partitioning_lab

```
sensor_type | sensor_id
------------+-----------
temperature |         0
temperature |         1
temperature |         2
temperature |         3
temperature |         4
temperature |         5
```

... until 999

pes2ug23cs100@pes2ug23cs100: ~/CC Lab/partitioning_lab

```
temperature |       940
temperature |       941
temperature |       942
temperature |       943
temperature |       944
temperature |       945
temperature |       946
temperature |       947
temperature |       948
temperature |       949
temperature |       950
temperature |       951
temperature |       952
temperature |       953
temperature |       954
temperature |       955
temperature |       956
temperature |       957
temperature |       958
temperature |       959
temperature |       960
temperature |       961
temperature |       962
temperature |       963
temperature |       964
temperature |       965
temperature |       966
temperature |       967
temperature |       968
temperature |       969
temperature |       970
temperature |       971
temperature |       972
temperature |       973
temperature |       974
temperature |       975
temperature |       976
temperature |       977
temperature |       978
temperature |       979
temperature |       980
temperature |       981
temperature |       982
temperature |       983
temperature |       984
temperature |       985
temperature |       986
temperature |       987
temperature |       988
temperature |       989
temperature |       990
temperature |       991
temperature |       992
temperature |       993
temperature |       994
temperature |       995
temperature |       996
temperature |       997
temperature |       998
temperature |       999
(1000 rows)

(END)
```

RESULT:

- Range partitioning can lead to hot spots
- Hash partitioning ensures uniform distribution
- Scatter/Gather indexing increases query overhead
- Global indexing improves read performance but increases write cost

CONCLUSION:

- This experiment demonstrated different partitioning and indexing strategies in distributed databases.
- Range partitioning in PostgreSQL groups similar data together, which can lead to hot spots when many records fall into the same partition.
- Cassandra uses hash partitioning to distribute data evenly across nodes, avoiding such issues.
- Additionally, scatter/gather indexing requires scanning multiple partitions, whereas global indexing improves query efficiency at the cost of increased write overhead.
- Thus, there exists a trade-off between performance and complexity in partitioning and indexing strategies.